

net-04 - Introdução ao SDN (Software-defined Networking) e Overlay Networks

Contexto [↗](#)

Neste treinamento iremos abordar:

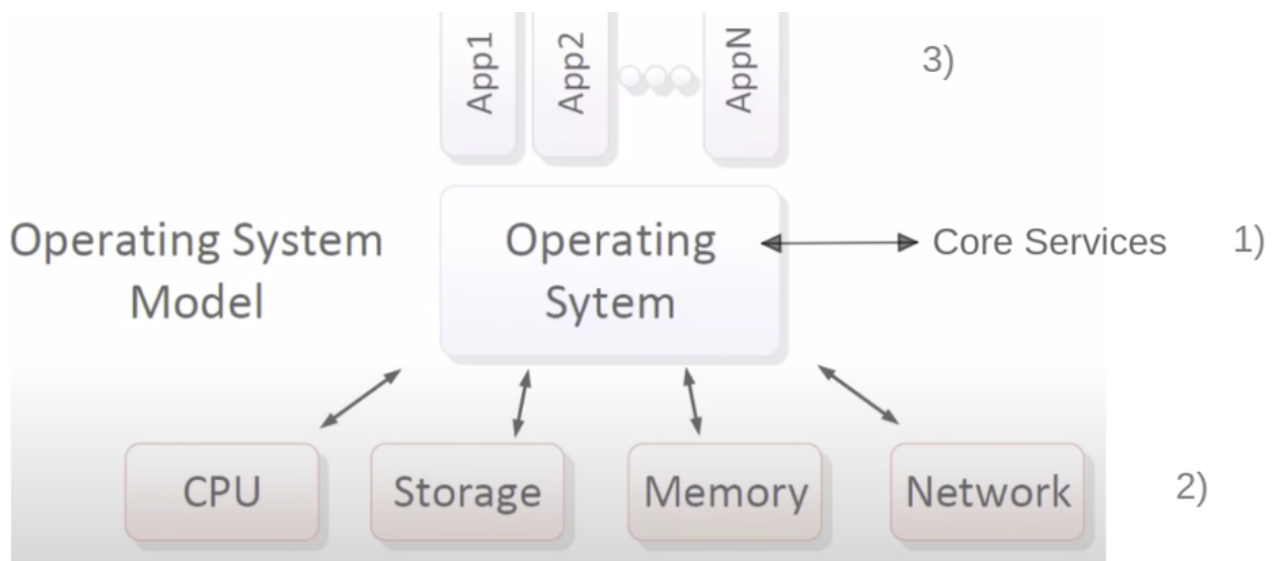
- Uma breve introdução ao SDN
- Deep dive no modelo de SDN
- Conhecendo Overlay Networks
- Explorando o Geneve

Introdução ao SDN [↗](#)

SDN (Software-Defined Networking) é um modelo de arquitetura de rede que adiciona uma camada de abstração para dispositivos que executam funções de rede (switches, routers, hypervisors, etc.), para gerenciá-los globalmente e de forma coerente.

Os dois principais objetivos do SDN é ser **Open** e **programável**.

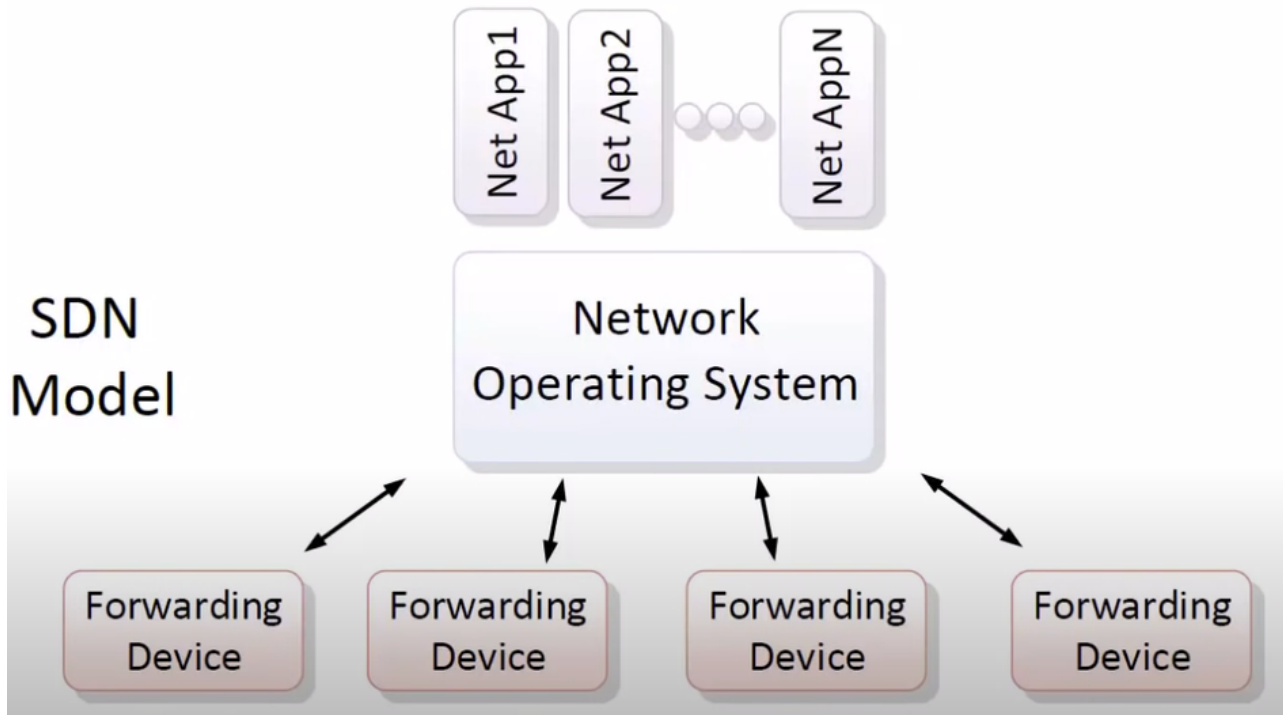
Fazendo uma analogia ao modelo de Sistema Operacional, temos o seguinte:



- Podemos dividir um servidor em 3 camadas: 1) Sistema Operacional e os Core Services, 2) Hardware e 3) Aplicações
- O Sistema Operacional atua como middleman gerenciando os acessos das Aplicações(North) ao Hardware. Em conjunto, os Core Services no nível de South, são responsáveis em gerenciar o Hardware em um nível mais baixo

No modelo de SDN, temos o seguinte:

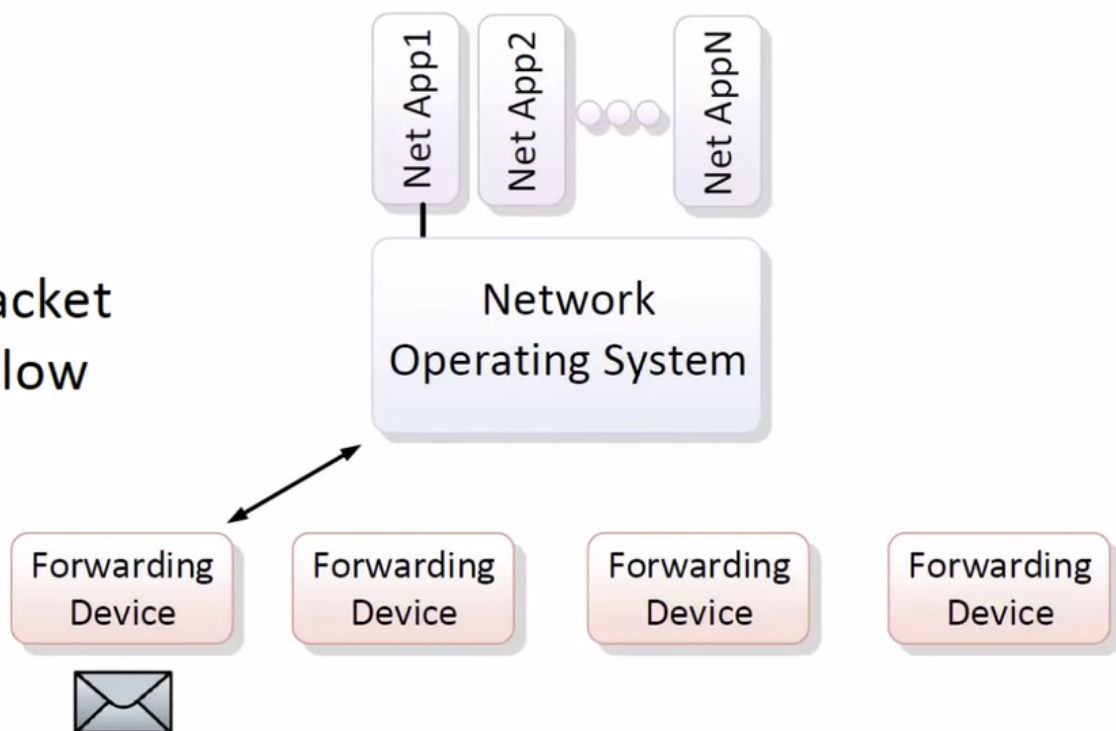
SDN Model



- Esse modelo é similar ao do Sistema Operacional, mas ao invés do Sistema Operacional convencional temos o Network Operating System que é chamado de SDN Controller
- A SDN Controller possui Core Services, que tem como responsabilidade prover uma interface programável para as aplicações (nível North) e também uma interface (nível South) com os dispositivos de rede (físico ou software) para programar o Forwarding Plane(conhecido também como Data Plane)

Seguindo com o modelo de SDN, **o que ocorre quando um pacote entra em uma porta de um dispositivo de rede físico ou software (ex: OVS)?**

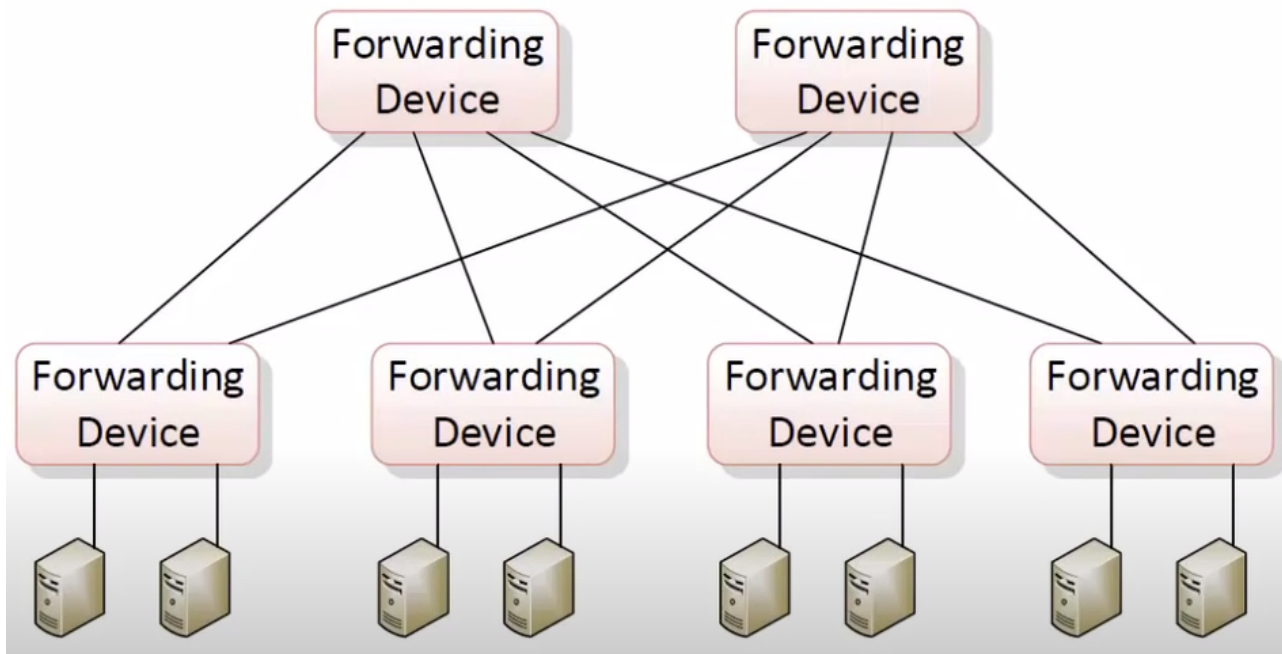
Packet Flow



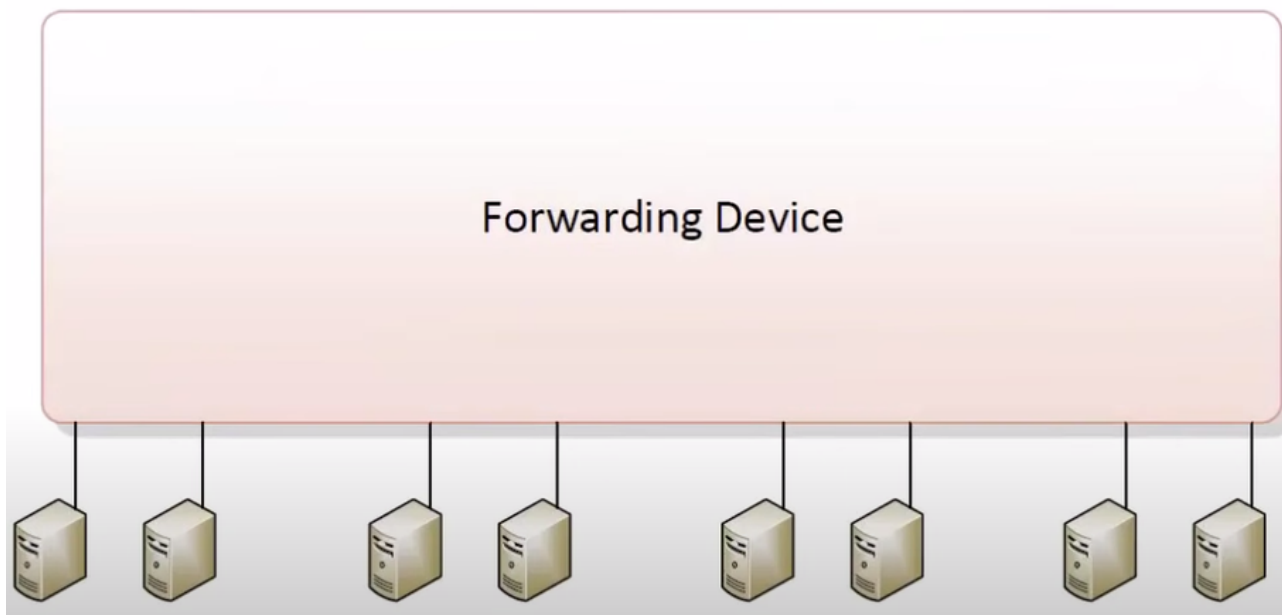
- Se o dispositivo de rede já tiver os flows instalados(fast path), o mesmo irá tomar decisões do que fazer com o pacote conforme o que está programado
- Caso não tenha os respectivos flows, o mesmo irá consultar a SDN Controller (slow path) que irá retornar com os flows para serem programados.

No modelo de SDN, podemos entender que existe uma camada que centraliza (SDN Controller) as decisões de como a rede irá se comportar e que possui uma visão global de toda a rede.

Um ponto muito vantajoso do modelo de SDN é que as aplicações externas não precisam saber como a rede fisicamente está disposta como abaixo:



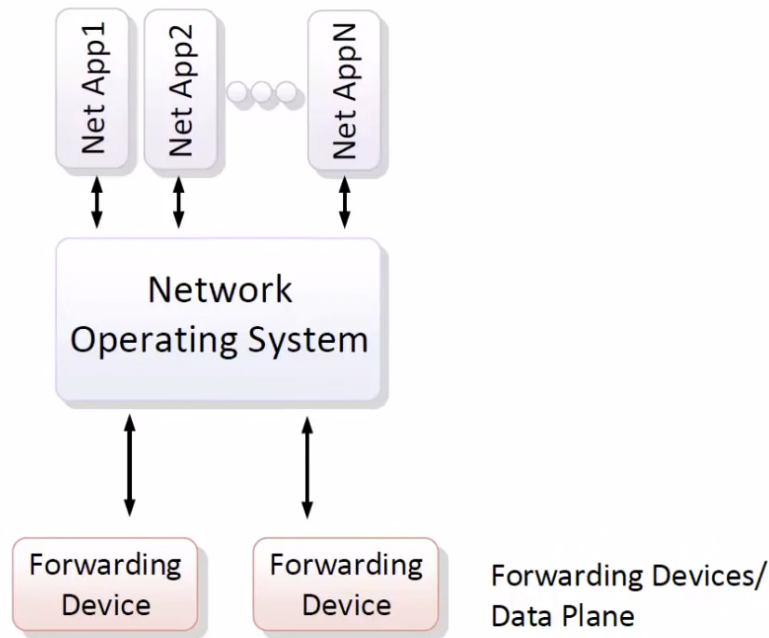
O modelo de SDN cria uma abstração para as aplicações externas enxergarem a rede como se fosse um único grande dispositivo de rede:



Deep dive no modelo de SDN [🔗](#)

Entrando em mais detalhes do modelo de SDN, iniciando pela camada mais baixa do modelo temos os dispositivos de rede com o Forwarding Plane ou Data Plane.

Os dispositivos de rede podem ser switches tradicionais desde que suportem uma interface programável como o Openflow/OVSDB/NETCONF ou podem ser como o OVS (Open vSwitch):



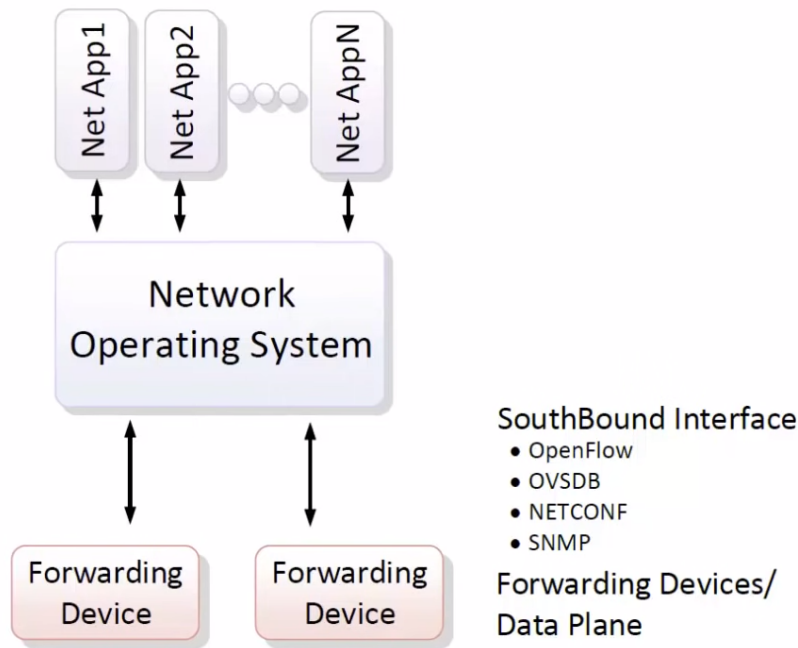
A SDN Controller necessita de uma maneira de se comunicar com o Forwarding Plane dos dispositivos de rede.

Várias informações entre a SDN Controller e o dispositivo de rede precisam ser trocadas, entre as principais temos:

- como manipular certo pacote que chegou em uma porta do dispositivo de rede
- notificação de portas que mudam de status up para down ou vice-versa
- prover estatísticas como counters de pacotes nas portas

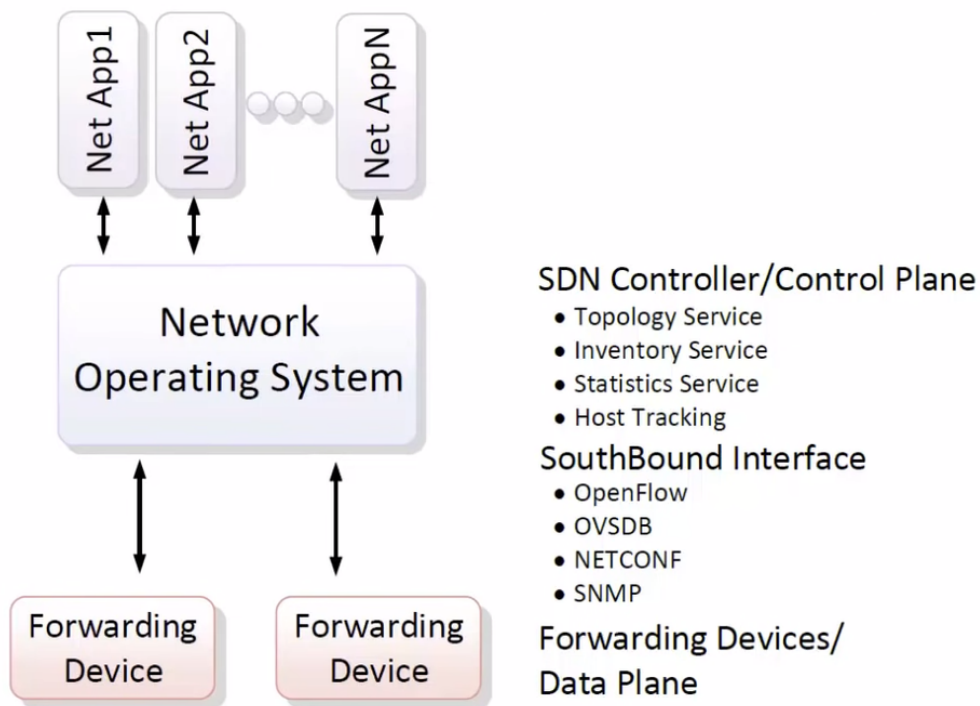
Toda essa troca de informação acontece via interface Southbound. O protocolo mais comum usado na Southbound é o Openflow, para mais informações sobre o Openflow verifique nosso treinamento 02.

Outra alternativa é o OVSDB, que vem sendo utilizado para gerenciar a database do OVS em switches e também em hypervisors:



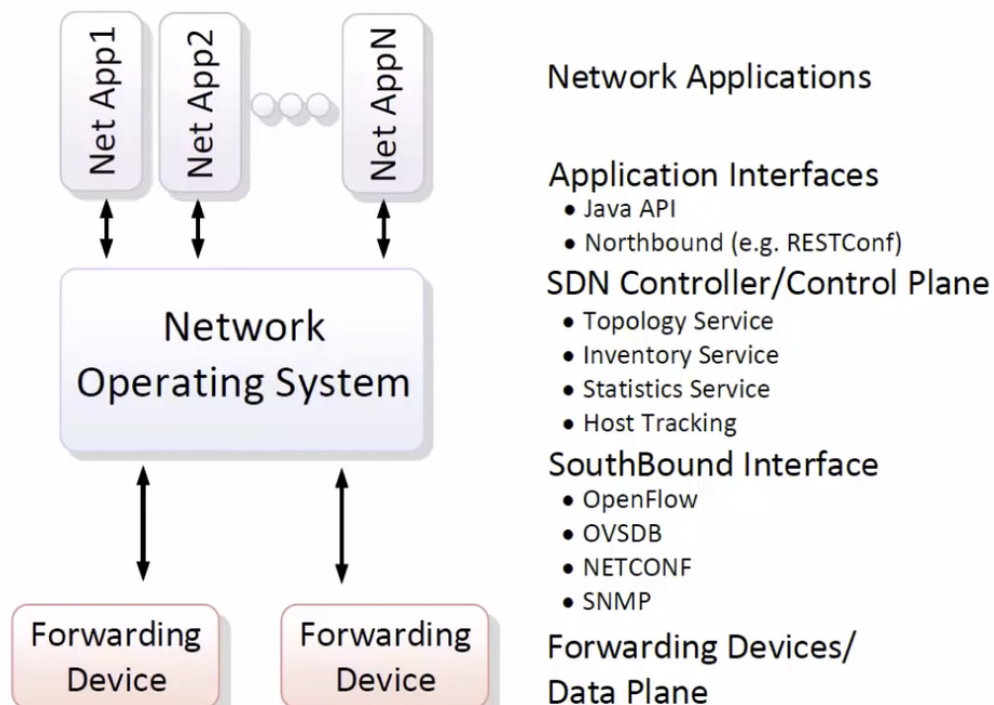
O Network Operating System também conhecido como SDN Controller, tipicamente executará os serviços que incluem:

- **Topology Service** - Determina como os dispositivos de rede se conectam entre si, utilizado para construir uma topologia baseado em protocolos de discovery como o LLDP
- **Inventory Service** - É usado para fazer um track de todos os dispositivos de rede e armazenar informações sobre os mesmos como versão do Openflow suportada e features que são suportadas
- **Statistics Service** - Pode ser empregado para ler as informações de counters das portas
- **Host Tracking** - Utilizado para descobrir onde os endereços IPs e como Mac Address estão localizados na rede. Muito importante no processo de criação, remoção ou live migration de VMs entre hypervisors



A SDN Controller deve prover uma interface Northbound para as aplicações externas e criar uma abstração conforme mencionado anteriormente para representar a rede física como um grande dispositivo de rede.

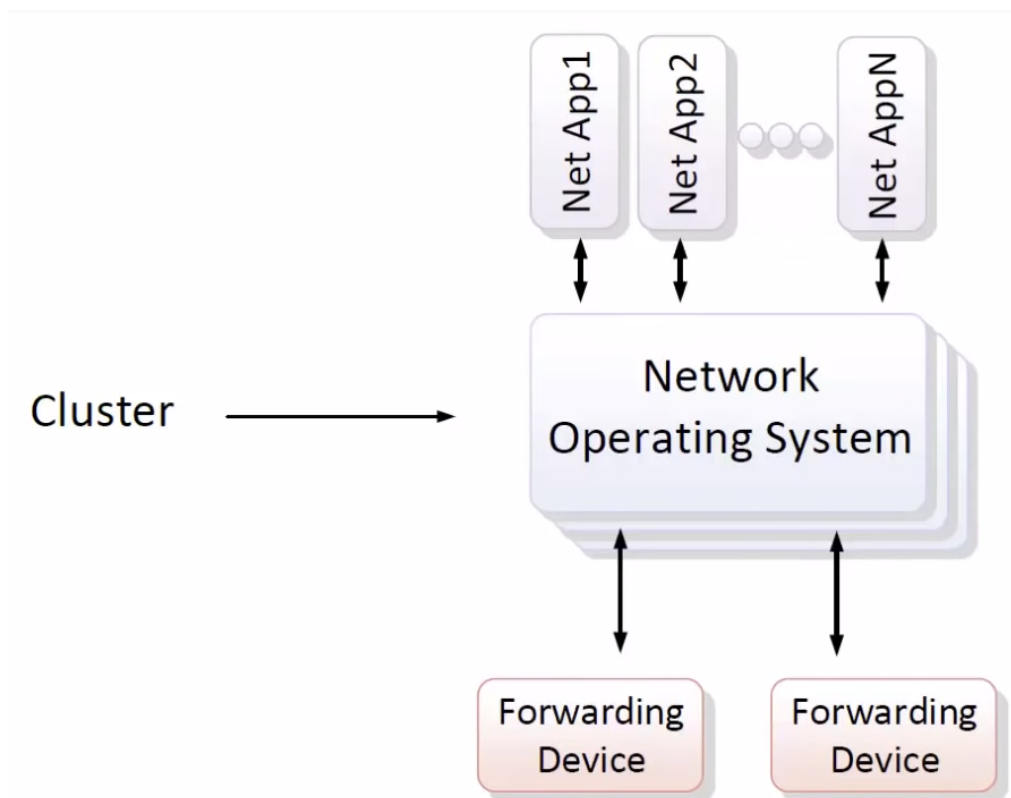
Através do mecanismo de Rest API ou outros meios, a interface de Northbound é apresentada para a aplicação externa que poderá programar a rede conforme sua necessidade:



Conforme contextualizado anteriormente, a SDN Controller é logicamente centralizada e é importante entender que logicamente é diferente de fisicamente.

Em um ambiente de produção, a SDN Controller não é fisicamente centralizada e não depende de um único ponto de falha.

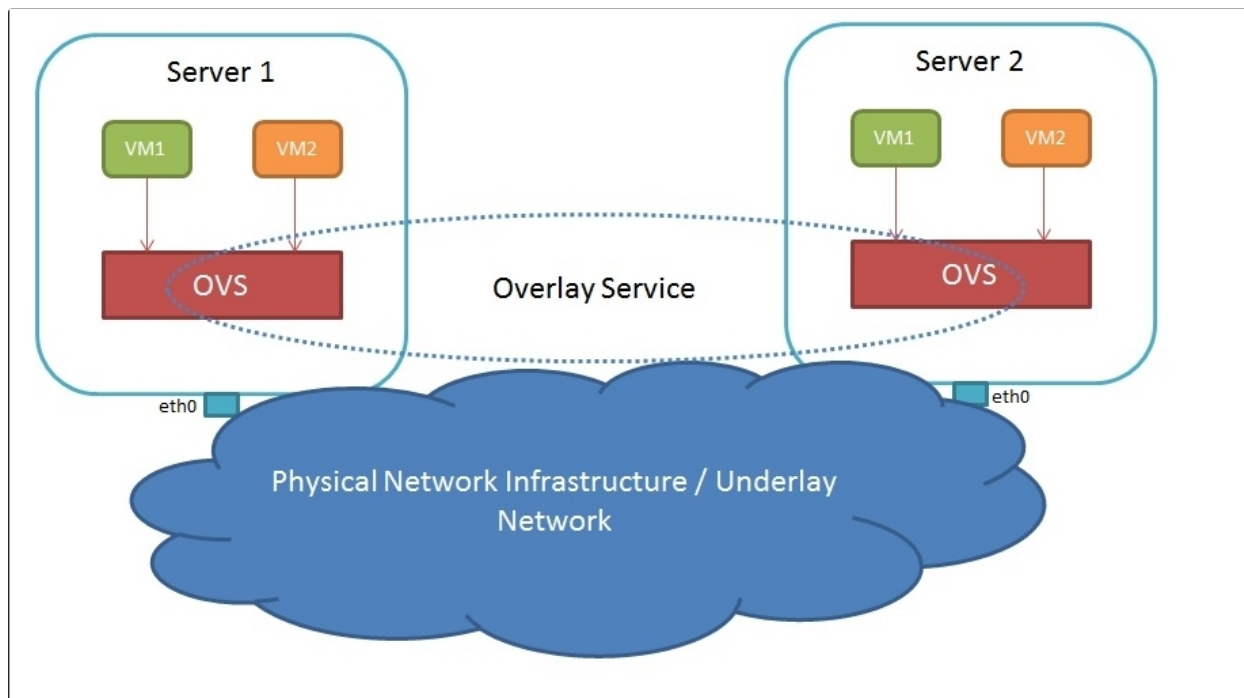
Diferentemente disso, a SDN Controller pode ser implementada em alta disponibilidade e ser logicamente distribuída para formar um cluster:



Conhecendo Overlay Networks [↗](#)

Antes de entrar em mais detalhes sobre o Overlay, primeiramente vamos apresentar as diferenças entre Underlay e Overlay:

- **Underlay** - representa a infraestrutura física de rede e tudo que é conectado nela (hypervisors e etc)
- **Overlay** - é uma abstração de rede criada sobre o Underlay que tem como intuito criar conectividade L2 e L3 para workloads



O Overlay se utiliza do encapsulamento dos pacotes, para ficar mais claro, fazendo uma analogia seria um envelope dentro do outro.

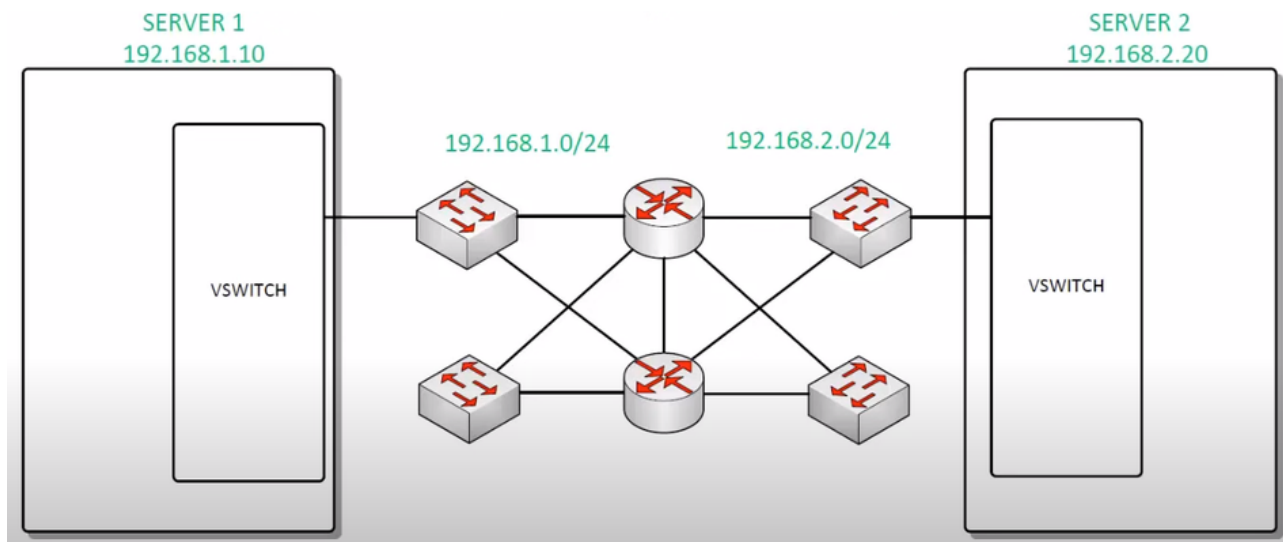
Nessa analogia, o envelope interno é o pacote de rede gerado pelos workloads que estão no Overlay, e o envelope externo representa um pacote no Underlay que fez o encapsulamento do envelope interno com um protocolo de encapsulamento.

Existem vários protocolos de encapsulamento para diferentes casos de uso:

- GRE
- IPSEC
- MPLS
- VXLAN
- STT
- GENEVE

Os protocolos de encapsulamento mais comuns para ambientes de Cloud são o VXLAN e o mais recente adotado o GENEVE. Essa abordagem de encapsulamento de pacotes é conhecida também como tunneling.

Voltando um pouco ao Underlay, vamos utilizar o exemplo abaixo para explicar como criamos conectividade de workloads para o Overlay:

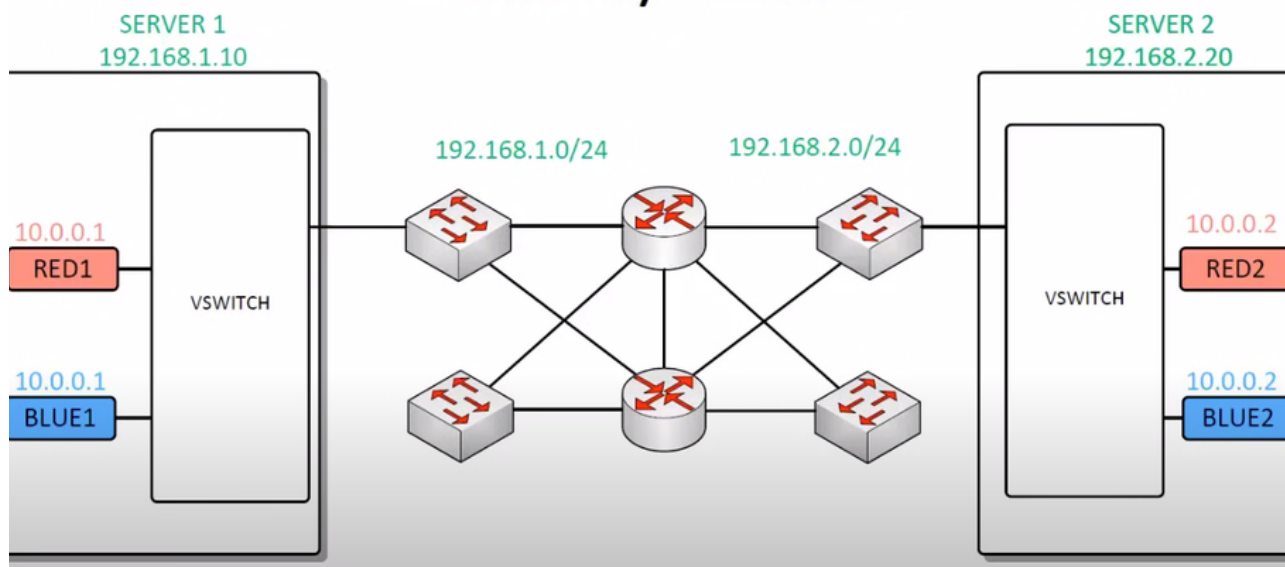


No centro desse diagrama temos um conjunto de equipamentos de rede que conectam dois servidores que fazem o papel de hypervisors com o OVS e também esses equipamentos de rede possuem logicamente a subnet 192.168.1.0/24 no lado esquerdo e 192.168.2.0/24 no lado direito. O hypervisor **SERVER1** está conectado na subnet 192.168.1.0/24 e o **SERVER2** está conectado na subnet 192.168.2.0/24 e ambos servidores possuem conectividade via Underlay.

O Overlay pode ser utilizado nessa topologia de rede para resolver várias limitações de uma solução de rede tradicional como:

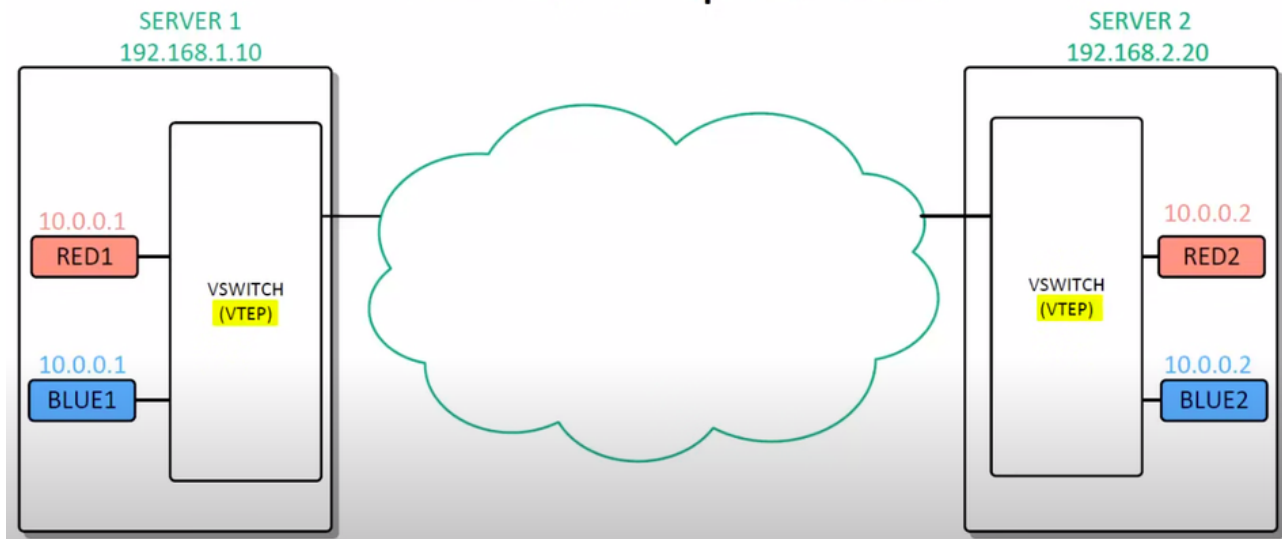
- Criar conectividade L2 dos workloads entre hypervisors que estão em pontos diferentes do Underlay
- Separação lógica do Underlay com redes virtuais
- O multitenancy pode ser implementado, podemos ter redes virtuais com o mesmo endereçamento de rede
- Separação lógica do Underlay e dessa forma criando uma camada de segurança
- Não existe limitação lógica quanto a domínios L2 (VLANs)
- Minimiza o forward table dos switches do Underlay

No exemplo abaixo, temos duas cores de workloads que representam os tenants **RED** e **BLUE**:

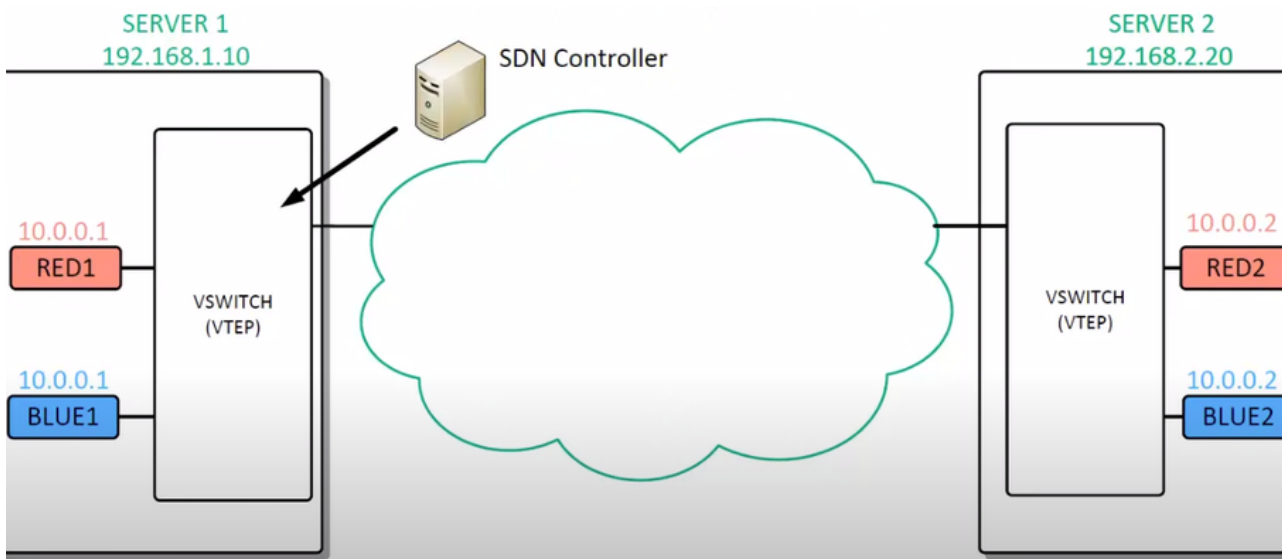


Ambos os tenants possuem os mesmos endereçamento de rede, ou seja estão com **Overlapping**.

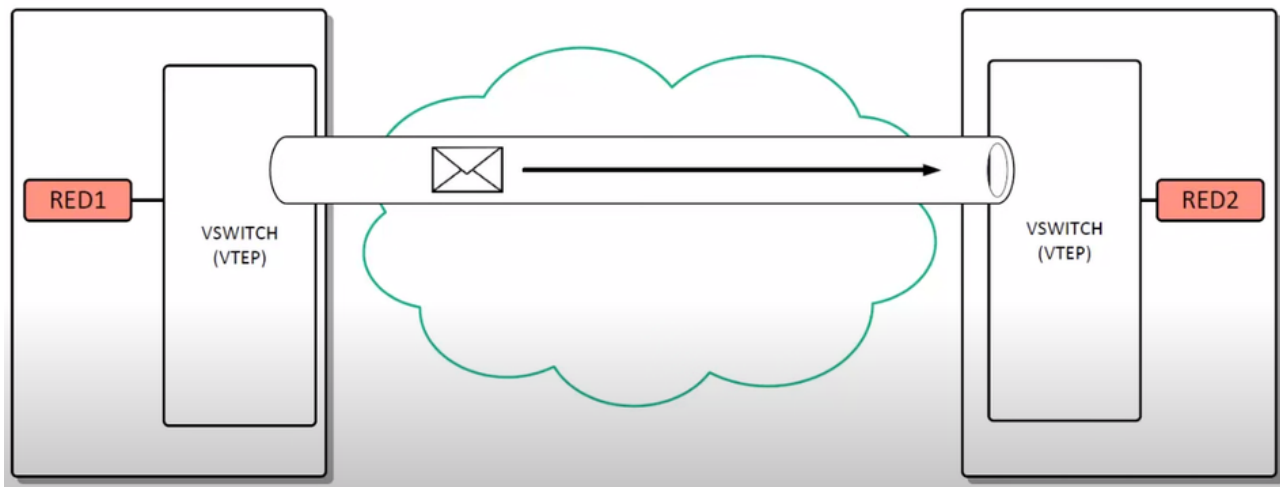
Para o Overlay funcionar corretamente, teremos que permitir que os hypervisors tenham conectividade de rede entre eles via Underlay e que o OVS faça o papel de **VTEP** (Virtual tunnel End Point):



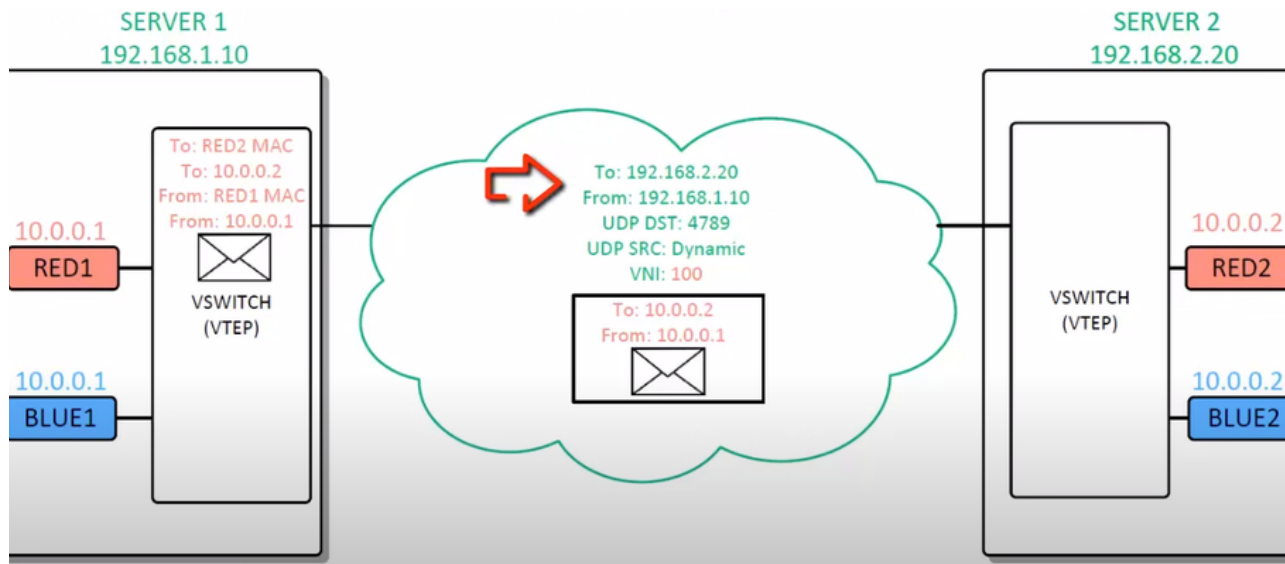
A configuração necessária para que o OVS faça o papel de VTEP e crie a abstração do Overlay, pode ser realizada de forma manual ou por uma SDN Controller via Openflow:



Conforme o que foi programado pela SDN Controller via Openflow, o OVS pode tomar a decisão de criar um tunnel usando um protocolo de encapsulamento (ex: VXLAN) entre os hypervisors para poder encaminhar os pacotes:



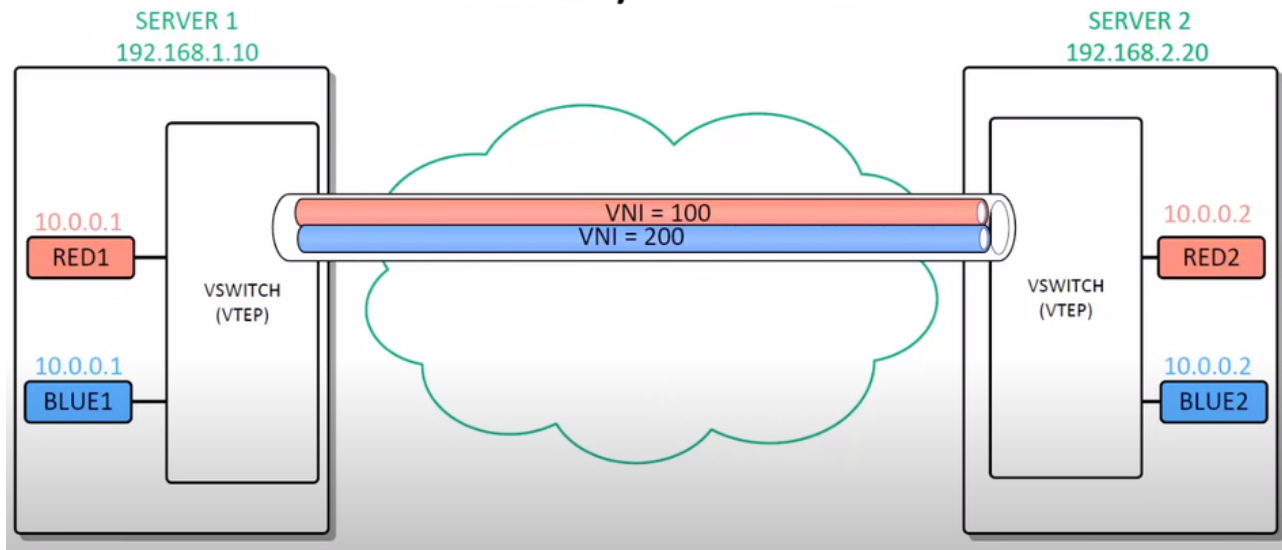
O encapsulamento irá ocorrer: o pacote do overlay será encapsulado em um pacote do Underlay:



Analisando o diagrama acima:

- quando o workload **RED1** envia um pacote para o workload **RED2**, o OVS adiciona uma camada de encapsulamento com a identificação que representa aquela rede virtual do Overlay e também do hypervisor de destino
- a rede Underlay apenas visualiza o header externo do pacote e faz o encaminhamento do pacote até o hypervisor de destino
- ao chegar no hypervisor de destino, o OVS irá analisar o pacote e realizar o processo inverso de desencapsulamento e entregar o pacote para o workload **RED2** de acordo com a identificação da rede virtual

Entrando em mais detalhes sobre a identificação da rede virtual, cada uma possui um **VNI** que significa Virtual Network Identifier. Para VXLAN e GENEVE, temos disponíveis até 16 milhões de VNIs e portanto podemos ter uma grande escala de redes virtuais.



Um ponto muito importante para entender sobre os protocolos de encapsulamento é que eles adicionam uma camada de overhead:

- VXLAN - 50 bytes
- GENEVE - 58 bytes

Portanto, as interfaces de rede dos hypervisors devem considerar o uso de Jumbo Frame, ou seja, alterar o MTU para algo maior considerando o overhead das aplicações. O comum para datacenters é usar MTU 9000 nos hypervisors e algo maior nos switches do Underlay (9216).

Explorando o Geneve [↗](#)

O **GENEVE** (Generic Network Virtualization Encapsulation) é um protocolo de encapsulamento que tem a habilidade de encapsular qualquer tipo de tráfego de rede.

Vale mencionar o motivo principal de ter sido criado o GENEVE já que o VXLAN é um protocolo bem difundido entre vários fabricantes de rede.

O VXLAN é um protocolo de encapsulamento muito utilizado e que tem em sua arquitetura uma limitação quanto a extensões de features. Para evitar problemas, os fabricantes foram incorporando modificações no protocolo porém sem padronização(standard) e gerando incompatibilidade no cenário multi-vendor.

No GENEVE, foi criado um mecanismo para permitir criar extensões e dessa forma possibilitando aos fabricantes poderem criar features e não quebrar os padrões (standard). Como exemplo hipotético, imagine inserir no header do pacote a identificação de um usuário ao invés de inserir no payload de dados.

Algo que o GENEVE pode inserir em seu header que o VXLAN não consegue é a informação das portas de input e output para os flows, que é bem mais vantajoso ao contrário do VXLAN que tem que calcular de outra forma.

Geneve Update

MAC
IP
UDP
Geneve
Options
Inner Eth
Inner IP
Inner L4
Payload

```

+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|Ver| Opt Len |O|C|   Rsvd. |           Protocol Type           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Virtual Network Identifier (VNI)           |   Reserved   |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Variable Length Options           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

Geneve is now supported in the following categories

Software	Linux, OVS, OVN
NICs	Intel, Broadcom, Mellanox, Netronome
Switch ASICs	Broadcom, Mellanox, Cavium, Centec
Monitoring Tools	wireshark, tcpdump, libcap

Verificando os headers de um pacote, existe um header UDP indicando que a porta de serviço é a 6081 e o header com os campos do GENEVE:

Outer UDP Header:

```

+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Source Port = xxxx           |           Dest Port = 6081           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           UDP Length           |           UDP Checksum           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

Geneve Header:

```

+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|Ver| Opt Len |O|C|   Rsvd. |           Protocol Type           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Virtual Network Identifier (VNI)           |   Reserved   |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           Variable Length Options           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

Mais detalhes do GENEVE serão abordados no treinamento 05

Conclusão [↗](#)

Neste treinamento foi possível entender o que é SDN, como funciona os protocolos de Overlay e explicamos um pouco sobre o Geneve.